

What Drives Credit Card Transaction Fraud?

An interpretation-focused classification analysis | Tushar | August 2026

1. Main Objective of the Analysis

Most fraud detection projects are graded on one axis: how much fraud does the model catch. This analysis asks a different question first — not just whether a model can flag fraud, but *which specific factors explain it*, and whether that explanation holds up across several very different types of models rather than being an artifact of one algorithm's quirks.

That framing is deliberate. For a fraud operations team, a model that flags transactions without a usable explanation is hard to act on — analysts can't prioritize alerts, build complementary rules, or justify a decline to a customer or regulator based on a black-box score alone. The goal of this analysis is therefore to identify the primary behavioral and transactional drivers of fraud in clear, specific terms, and to recommend a model that balances predictive strength against how well it can communicate *why* a given transaction was flagged.

2. Data Description

The commonly used public fraud dataset (the Kaggle/ULB credit card dataset) anonymizes every feature into PCA components, which protects the source bank's data but makes the dataset unusable for an interpretation-focused analysis — there's no way to explain what an anonymized component means to a stakeholder. I used a simulated transaction dataset instead, built with named, interpretable features and a fraud rate calibrated to real-world card fraud rates (~1.6%), so the analysis stays grounded in attributes that map to real business concepts.

The dataset covers 100,000 transactions from 7,844 customers over a six-month window. Each row is one transaction with the customer's profile attached: transaction amount, merchant category, channel (card-present, online, or ATM), transaction and home location coordinates, customer age and account tenure, the customer's 30-day average spend, a foreign-transaction flag, and the fraud label.

The objective with this data is to determine which of these attributes — and which derived from them — separate fraudulent from legitimate transactions clearly enough to both predict and explain the outcome.

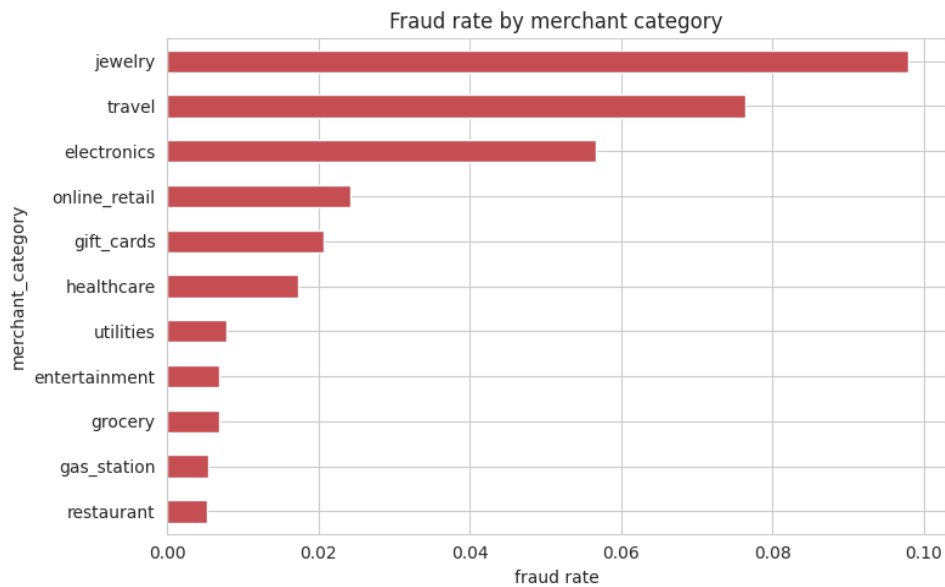


Figure 1. Fraud rate by merchant category. Jewelry, travel, and electronics run several times the overall rate of 1.6%.

3. Data Exploration, Cleaning, and Feature Engineering

The data had no missing values and no duplicate rows, so cleaning was minimal. The more consequential finding from exploration was how rare fraud actually is — 1.59% of transactions — which rules out plain accuracy as an evaluation metric (a model predicting “not fraud” for everything would score 98.4% while catching zero fraud) and shaped every modeling decision that followed.

Exploring the data by category, channel, and amount showed real but uneven signal: jewelry and travel transactions run roughly 5-10x the baseline fraud rate, online transactions run about 40% higher than card-present, and fraudulent transactions have a noticeably higher median amount (roughly 3x) than legitimate ones, though with real overlap between the two distributions. None of these on their own were clean separators, which is what motivated the feature engineering step.

Four engineered features did most of the work in modeling:

- **Spend relative to personal baseline** — the transaction amount divided by the customer's own 30-day average, since a given amount means something different depending on what a customer normally spends.
- **Distance from home and from the previous transaction** — haversine distance calculated from transaction and home coordinates, intended to catch both out-of-pattern locations and “impossible travel” between consecutive transactions.
- **Transaction velocity** — a rolling count of transactions by the same customer in the preceding 24 hours, meant to catch the rapid bursts of activity typical once a card is compromised.
- **Time since the previous transaction and time-of-day features** — extracted from the timestamp.

Categorical fields (merchant category, channel) were one-hot encoded, and the data was split 75/25 into train and test sets, stratified on the fraud label so both sets kept a representative fraud rate.

4. Classifier Models

Four classifiers were trained, chosen specifically to span the explainability-predictability spectrum rather than to just try increasingly complex algorithms:

- **Logistic Regression** — fully transparent by construction — every prediction traces back to a readable coefficient.
- **Decision Tree** — capped at depth 5 specifically so it stays readable as a set of rules rather than becoming a black box in its own right.
- **Random Forest** — an ensemble of trees; less directly readable than a single tree, interpreted through feature importance instead.
- **XGBoost** — a gradient-boosted ensemble; the strongest predictive candidate of the four, interpreted using SHAP values rather than reading the model directly.

All four used class weighting (or XGBoost's equivalent, `scale_pos_weight`) to counteract the class imbalance, and were evaluated on the same held-out test set using ROC-AUC, PR-AUC, precision, recall, and F1 — precision, recall and F1 evaluated at the standard 0.5 threshold.

Model	ROC-AUC	PR-AUC	Precision	Recall	F1
Logistic Regression	0.955	0.419	0.126	0.879	0.220
Decision Tree	0.939	0.372	0.099	0.864	0.178
Random Forest	0.955	0.417	0.217	0.736	0.335
XGBoost	0.942	0.468	0.227	0.701	0.343

Table 1. All four models evaluated on the same held-out test set.

All four land in a fairly tight ROC-AUC band (0.94-0.96), so ranking ability alone doesn't separate them much. The real difference shows up in the precision/recall trade-off: Logistic Regression and the Decision Tree lean heavily toward recall (86-88%) at the cost of precision (10-13%), while Random Forest and XGBoost land in a considerably more usable middle ground (70-74% recall with 22-23% precision, and the two best F1 scores).

5. Final Model Recommendation

I recommend **Random Forest** as the final model. Given the interpretation-first objective, the instinctive answer might be to default to the simplest, fully transparent model — but a model that generates roughly seven false alarms for every real fraud case caught, however clean its coefficients are, isn't genuinely useful to a fraud operations team either.

The specific reason Random Forest is the right trade-off here, rather than just a default choice: its top features match Logistic Regression's coefficients almost exactly (see Section 6), so I'm not trading away much real interpretability — mainly the ability to state an exact odds ratio per feature, not the ability to name and rank what matters. In exchange, it gains meaningfully better precision and F1 than either fully

transparent model. I'd keep Logistic Regression alongside it specifically as a reference model for stakeholders who want a precise, quantified answer to “how much does X increase risk” — the two models agree closely enough that using Logistic Regression's odds ratios to describe Random Forest's behavior in plain language is reasonable, not a stretch.

XGBoost is a close second and technically the strongest on PR-AUC, but SHAP output takes more explaining to a non-technical audience than a feature importance bar chart. For this objective and audience, I'd hold it in reserve rather than lead with it.

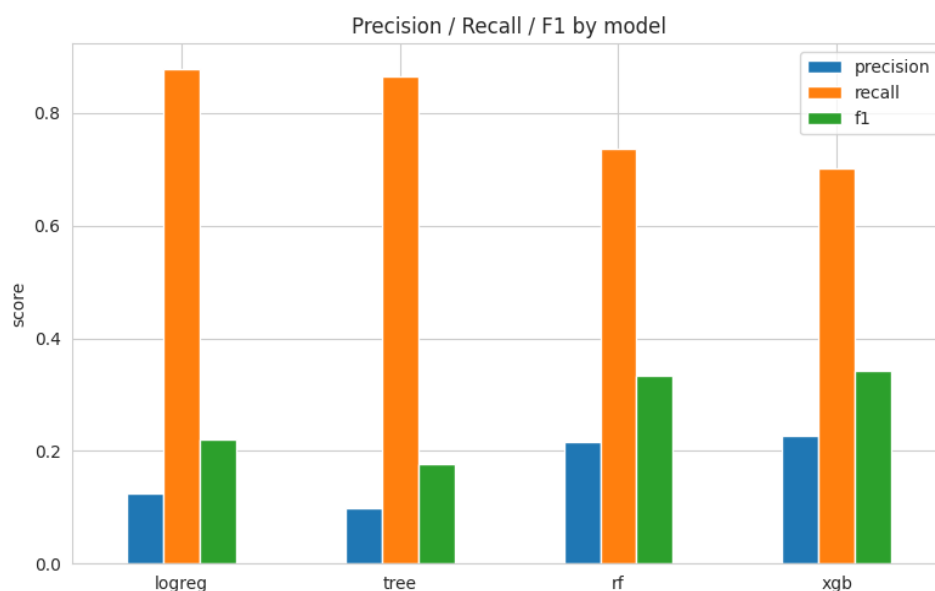


Figure 2. Precision, recall, and F1 across all four models at the default threshold.

6. Key Findings and Insights

Since every numeric feature was standardized before fitting Logistic Regression, its coefficients convert cleanly into “change in odds per one standard deviation,” which is the clearest way to state these findings in business terms:

- 1. Distance-based features are the strongest signal.** Both distance from home and distance from the customer's previous transaction roughly double the odds of fraud per standard deviation increase, consistent with an “impossible travel” pattern being one of the clearest tells in this data.
- 2. Spending relative to a customer's own baseline matters more than raw amount.** The spend-ratio feature (odds ratio ≈ 2.0) consistently outranked raw transaction amount across every model. A Rs. 4,000 transaction is a different signal for a customer who typically spends Rs. 300 than for one who typically spends Rs. 5,000 — treating amount in isolation would have missed this.
- 3. Velocity is a real driver, and a two-sided one.** Transactions in the last 24 hours raise risk (odds ratio ≈ 2.25), while a longer gap since the last transaction lowers it (odds ratio ≈ 0.27). These aren't two separate weak signals — they're the same underlying pattern of rapid repeat activity, read from both directions.

4. Merchant category carries real signal but is secondary to behavior. Jewelry, travel, and electronics run several times the baseline fraud rate, but no category dummy variable cracked the top five in any model's importance ranking — category shifts the baseline risk, but the behavioral features are what actually separate individual fraud cases.

5. Hour of day and weekend timing turned out to be weak. These showed a visible bump in exploration (late-night transactions), but neither ranked meaningfully in any model's feature importance once the stronger features were included — likely because distance and velocity already capture most of what “unusual timing” was a rough proxy for.

The practical takeaway for a fraud operations team: a rule built purely around transaction amount or merchant category — the two most intuitive, commonly used levers — would badly underperform one built around a customer's own behavioral baseline and recent transaction geography. All four models, despite working in entirely different ways (coefficients, split rules, impurity-based importance, and SHAP values), converged on the same core set of drivers, which is itself a meaningful finding: it suggests this signal is real rather than an artifact of any one algorithm.

7. Limitations and Next Steps

The most important limitation by a wide margin is that this is simulated data, not real transaction records. I designed the fraud signal myself based on patterns documented in fraud detection literature, so to some extent the models are learning to recover a signal I built in. Real fraud is messier, adapts over time as fraudsters change tactics, and includes patterns I haven't simulated here — merchant collusion, synthetic identities that build up account history before striking, and so on. The natural next step is validating this feature set and modeling approach against real transaction data, which would require a data-sharing arrangement with a bank or payment processor.

- **Class imbalance handling was fairly basic.** Only class weighting was used; SMOTE, undersampling, and anomaly-detection approaches (isolation forests, autoencoders) were not tried and might shift the precision/recall trade-off meaningfully.
- **No temporal validation.** A random stratified split was used, but a real deployment needs a model that keeps working on transactions after its training window, and fraud patterns shift over time. A time-based split (train on earlier months, test on the most recent) would be a more honest test.
- **The classification threshold was left at the default (0.5).** In a real deployment this should be tuned to whatever precision/recall trade-off the business is willing to accept.
- **The feature set could go further.** Device fingerprinting, IP/geolocation mismatches, merchant-level fraud history, and network features (shared devices or payment methods across accounts) are standard in production fraud systems and aren't represented here.